Mysql笔记

初始Mysql

介绍

mysql数据库是为了方便对数据进行存储以及管理的。

术语

```
数据库：数据库是一些关联表的集合
数据表：表是数据的矩阵。
列：一列包含了相同类型的数据。
行：一行是一组相关的数据。
主键：主键是唯一的，可以由一列或者多列构成关键字。
外键：外键是用于关联两个表。
```

特性

```
原子性是指事务包含的所有操作要么全部成功，要么全部失败回滚。失败回滚的操作事务，将不能对事务有任何影响。

一致性是指事务必须使数据库从一个一致性状态变换到另一个一致性状态，也就是说一个事务执行之前和执行之后都必须处于一致性状态。

隔离性是指当多个用户并发访问数据库时，比如同时访问一张表，数据库每一个用户开启的事务，不能被其他事务所做的操作干扰
(也就是事务之间的隔离)，多个并发事务之间，应当相互隔离。

持久性是指事务的操作，一旦提交，对于数据库中数据的改变是永久性的，即使数据库发生故障也不能丢失已提交事务所完成的改变。
```

安装

windows 上安装Mysql

下载地址

数据类型

类型	大小	用途
TINYINT	1 byte	小整数值
SMALLINT	2 bytes	大整数值
MEDIUMINT	3 bytes	大整数值
INT或INTEGER	4 bytes	大整数值
BIGINT	8 bytes	极大整数值
FLOAT	4 bytes	单精度 浮点数值
DOUBLE	8 bytes	双精度 浮点数值
DECIMAL	对DECIMAL(M,D)，如果M>D，为M+2否则为D+2	小数值

字符串类型	0-255 bytes	定长字符串
VARCHAR	0-65535 bytes	变长字符串
TINYBLOB	0-255 bytes	不超过 255 个字符的二进制字符串
TINYTEXT	0-255 bytes	短文本字符串
BLOB	0-65 535 bytes	二进制形式的长文本数据
TEXT	0-65 535 bytes	长文本数据
MEDIUMBLOB	0-16 777 215 bytes	二进制形式的中等长度文本数据
MEDIUMTEXT	0-16 777 215 bytes	中等长度文本数据
LONGBLOB	0-4 294 967 295 bytes	二进制形式的极大文本数据
LONGTEXT	0-4 294 967 295 bytes	极大文本数据

####

操作数据库

进入mysql

```
mysql -u root -p

-u root 其中 root为用户名
```

显示数据库

```
show databases;
```

删除数据库

```
drop database 数据库名;
```

创建数据库

```
create database 数据库名 charset=utf8;

或者

# utf8
create database 数据库名 default character set utf8 collate utf8_general_ci;

# utf8mb4
create database 数据库名 default character set utf8mb4 collate utf8mb4_unicode_ci;
```

选择数据库

```
use 数据库名;
```

显示所有数据表

```
show tables;
```

数据管理

创建数据表

```
create table 表名 (
    列名 列类型 AUTO_INCREMENT,
    列名 列类型 not null,
    PRIMARY KEY(列名)
)ENGINE=InnoDB DEFAULT CHARSET=utf8;

###  判断表是否存在，不存在则创建
create table if not exists 表名(
列名 列类型 AUTO_INCREMENT,
列名 列类型 not null,
PRIMARY KEY('列名')
)ENGINE=InnoDB DEFAULT CHARSET=utf8;



column_name data_type[size] [NOT NULL|NULL] [DEFAULT value]
[AUTO_INCREMENT]


如果你不想字段为 NULL 可以设置字段的属性为 NOT NULL， 在操作数据库时如果输入该字段的数据为NULL ，就会报错。
AUTO_INCREMENT定义列为自增的属性，一般用于主键，数值会自动加1。
PRIMARY KEY关键字用于定义列为主键。 您可以使用多列来定义主键，列间以逗号分隔。
ENGINE 设置存储引擎，CHARSET 设置编码。
DEFAULT值用于指定列的默认值。
```

外键约束

```
create table t1(
    t1_c1 列类型 AUTO_INCREMENT,
    t1_c2 列类型 not null,
    primary key (t1_c1)
)


create table t2(
    t2_c1 列类型 AUTO_INCREMENT,
    t2_c2 列类型 not null,
    primary key (t2_c1)
    foreign key (t2_c2) references t1(t1_c1)
)

通过foregin key连接两个表，其中t2_c2是用于标识t1表，t1_c1是t1表的id
```

指定多个外键约束

```
create table 表名
    (
        column1 datatype null/not null,
        column2 datatype null/not null,
        ...
        constraint 外键约束名 foreign key  (column1,column2,... column_n)
        references 外键依赖的表 (column1,column2,...column_n)
        on delete cascade--级联删除
    );
```

在创建表后增加外键约束

```
alter table 表名
    add constraint 外键约束名
    foreign key (column1, column2,...column_n)
    references 外键所依赖的表 (column1,column2,...column_n)
    on delete cascade ;--级联删除
```

删除数据表

```
drop table 表名;
```

插入数据

```
insert into 表名(列名1,,列名2,...,列名n) values (值1,值2,...,值n)
```

更新数据

```
update 表名 set 列名1=值1,列名2=值2 [where 条件]
```

删除数据

- 删除 `[指定条件的]` 数据

```
delete from 表名 [where 条件]
```

- 删除所有数据

```
truncate table 表名
```

修改数据

```
# 删除
alter table 表名 column 列名;

# 添加
alter table 表名 add column 字段名 字段类型

# 添加索引
ALTER TABLE 表名 ADD UNIQUE KEY 字段名 (`identity_card`);
ALTER TABLE 表名 ADD KEY 新字段名(多列字段名);
ALTER TABLE 表名 ADD FULLTEXT INDEX 新字段名 (`phone`);

# 修改
alter table 表名 modify 字段名 字段类型;
alter table 表名 change 字段名 新字段名 字段类型;
```

SQL查询数据

查询数据

```
select 列名1,列名2 from 表名[ where 条件] [LIMIT N] [OFFSET M] [distinct]

limit 返回的记录数
offset 偏移量
distinct 用于去除重复数据
```

别名

```
select 列名1 as 新列名1,列名2 as 新列名2 from 表名;


select 新表名1.列名 as 新列名1 from 表名1 [as] 新表名1,表名2 [as] 新表名2 where 新表名1.列名 == 新表名2.列名
```

where条件

下面的运算符可以在 WHERE 子句中使用：

运算符	描述
=	等于
!= / not	不等于
>	大于
<	小于
>=	大于等于
<=	小于等于
BETWEEN A and B	在某个范围内
LIKE	搜索某种模式
IN	指定针对某个列的多个可能值

模糊查询

```
select * from 表名 where 列名 like '%值%'

%是匹配多个字符
```

在 SQL 中，可使用以下通配符：

通配符	描述
%	替代 0 个或多个字符
_	替代一个字符
[charlist]	字符列中的任何单一字符
[^charlist] 或 [!charlist]	不在字符列中的任何单一字符

排序

```
select * from order by 列名 [ASC [DESC][默认 ASC]]

ASC:升序
DESC:降序
```

分组和过滤

group by:对select查询出来的结果集按照某个字段或者表达式进行分组，获得一组组的集合，然后从每组中取出一个指定字段或者表达式的值。

having： 用于对where和group by查询出来的分组经行过滤，查出满足条件的分组结果。它是一个过滤声明，是在查询返回结果集以后对查询结果进行的过滤操作。

```
select * from 表名
    where 条件
    group by 列名
    having 条件;
```

> 在分组的列上可以使用 COUNT, SUM, AVG,等函数。

where和视图，having子句作用于组。
where在分组和聚集计算之前选取输入行（因此，它控制哪些行进入聚集计算）， 而 having在分组和聚集之后选取分组的行。where子句不能包含聚合函数。

> **having一般跟在group by之后，执行记录组选择的一部分来工作的。**
> **where则是执行所有数据来工作的。**

子查询

```
select * from 表名1 where 列名1=(
    select 列名1 from 表名2（也可以为表名1）
    where 条件
)
```

可以嵌套下去...

联表查询

```
INNER JOIN（内连接,或等值连接）：获取两个表中字段匹配关系的记录。

LEFT JOIN（左连接）：获取左表所有记录，即使右表没有对应匹配的记录。

RIGHT JOIN（右连接）： 与 LEFT JOIN 相反，用于获取右表所有记录，即使左表没有对应匹配的记录。

FULL JOIN(全连接)：关键字只要左表（table1）和右表（table2）其中一个表中存在匹配，则返回行。就是结合了 LEFT JOIN 和 RIGHT JOIN 的结果


select * from 表名 inner join 表名.列名=表名.列名;
```

合并数据

UNION 操作符用于合并两个或多个 SELECT 语句的结果集。

请注意，UNION 内部的每个 SELECT 语句必须拥有相同数量的列。列也必须拥有相似的数据类型。同时，每个 SELECT 语句中的列的顺序必须相同。

```
SELECT column_name(s) FROM table1
UNION [ALL | DISTINCT]
SELECT column_name(s) FROM table2;
```

注释： UNION 结果集中的列名总是等于 UNION 中第一个 SELECT 语句中的列名。

- **DISTINCT:** 可选，删除结果集中重复的数据。默认情况下 UNION 操作符已经删除了重复数据，所以 DISTINCT 修饰符对结果没啥影响。
- **ALL:** 可选，返回所有结果集，包含重复数据。

mysql函数

MySQL 数值型函数

函数名称	作 用
ABS	求绝对值
SQRT	求二次方根
MOD	求余数
CEIL 和 CEILING	两个函数功能相同，都是返回不小于参数的最小整数，即向上取整
FLOOR	向下取整，返回值转化为一个BIGINT
RAND	生成一个0~1之间的随机数，传入整数参数是，用来产生重复序列
ROUND	对所传参数进行四舍五入
SIGN	返回参数的符号
POW 和 POWER	两个函数的功能相同，都是所传参数的次方的结果值
SIN	求正弦值
ASIN	求反正弦值，与函数 SIN 互为反函数
COS	求余弦值
ACOS	求反余弦值，与函数 COS 互为反函数
TAN	求正切值
ATAN	求反正切值，与函数 TAN 互为反函数
COT	求余切值

MySQL 字符串函数

函数名称	作 用
LENGTH	计算字符串长度函数，返回字符串的字节长度
CONCAT	合并字符串函数，返回结果为连接参数产生的字符串，参数可以使一个或多个
INSERT	替换字符串函数
LOWER	将字符串中的字母转换为小写
UPPER	将字符串中的字母转换为大写
LEFT	从左侧字截取符串，返回字符串左边的若干个字符
RIGHT	从右侧字截取符串，返回字符串右边的若干个字符
TRIM	删除字符串左右两侧的空格
REPLACE	字符串替换函数，返回替换后的新字符串
SUBSTRING	截取字符串，返回从指定位置开始的指定长度的字符换
REVERSE	字符串反转（逆序）函数，返回与原始字符串顺序相反的字符串

MySQL 日期和时间函数

函数名称	作 用
CURDATE 和 CURRENT_DATE	两个函数作用相同，返回当前系统的日期值
CURTIME 和 CURRENT_TIME	两个函数作用相同，返回当前系统的时间值
NOW 和 SYSDATE	两个函数作用相同，返回当前系统的日期和时间值
UNIX_TIMESTAMP	获取UNIX时间戳函数，返回一个以 UNIX 时间戳为基础的无符号整数
FROM_UNIXTIME	将 UNIX 时间戳转换为时间格式，与UNIX_TIMESTAMP互为反函数
MONTH	获取指定日期中的月份
MONTHNAME	获取指定日期中的月份英文名称
DAYNAME	获取指定日期对应的星期几的英文名称
DAYOFWEEK	获取指定日期对应的一周的索引位置值
WEEK	获取指定日期是一年中的第几周，返回值的范围是否为 0～52 或 1～53
DAYOFYEAR	获取指定日期是一年中的第几天，返回值范围是1~366
DAYOFMONTH	获取指定日期是一个月中是第几天，返回值范围是1~31
YEAR	获取年份，返回值范围是 1970～2069
TIME_TO_SEC	将时间参数转换为秒数
SEC_TO_TIME	将秒数转换为时间，与TIME_TO_SEC 互为反函数
DATE_ADD 和 ADDDATE	两个函数功能相同，都是向日期添加指定的时间间隔
DATE_SUB 和 SUBDATE	两个函数功能相同，都是向日期减去指定的时间间隔
ADDTIME	时间加法运算，在原始时间上添加指定的时间
SUBTIME	时间减法运算，在原始时间上减去指定的时间
DATEDIFF	获取两个日期之间间隔，返回参数 1 减去参数 2 的值
DATE_FORMAT	格式化指定的日期，根据参数返回指定格式的值
WEEKDAY	获取指定日期在一周内的对应的工作日索引

MySQL 聚合函数

函数名称	作用
MAX	查询指定列的最大值
MIN	查询指定列的最小值
COUNT	统计查询结果的行数
SUM	求和，返回指定列的总和
AVG	求平均值，返回指定列数据的平均值

MySQL 流程控制函数

函数名称	作用
IF	判断，流程控制
IFNULL	判断是否为空
CASE	搜索语句

数据库级别的MD5加密

- 创建数据库

```
create table testmd5(
    id int not null,
    name varchar(20) not null,
    pwd varchar(50) not null,
    primary key(id)
)engine=innodb default charset=utf8;

insert into testmd5 values (1,"胡桃","123456")

```

```
-- 将所有密码进行加密
update testmd5 set pwd=MD5(pwd)

-- 插入的时候进行加密
insert into testmd5 values (1,"胡桃",MD5("123456"))
```

事务ACID原则

什么是事务

事务包含的所有操作要么全部成功，要么全部失败回滚。

- **原子性**是指事务包含的所有操作要么全部成功，要么全部失败回滚。失败回滚的操作事务，将不能对事务有任何影响。
- **一致性**是指事务必须使数据库从一个一致性状态变换到另一个一致性状态，也就是说一个事务执行之前和执行之后都必须处于一致性状态。
- **隔离性**是指当多个用户并发访问数据库时，比如同时访问一张表，数据库每一个用户开启的事务，不能被其他事务所做的操作干扰(也就是事务之间的隔离)，多个并发事务之间，应当相互隔离。
- **持久性**是指事务的操作，一旦提交，对于数据库中数据的改变是永久性的，即使数据库发生故障也不能丢失已提交事务所完成的改变。

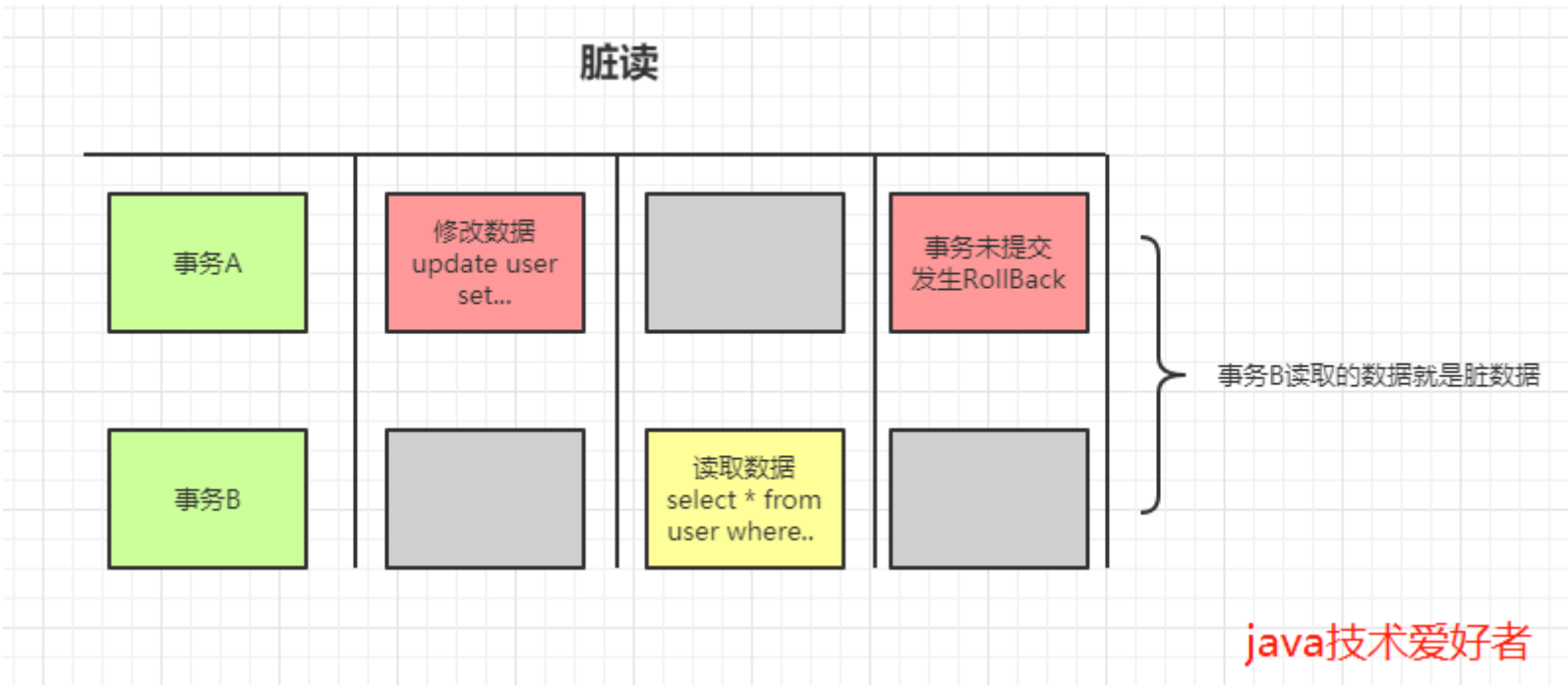
隔离所导致的问题

脏读

1、在事务A执行过程中，事务A对数据资源进行了修改，事务B读取了事务A修改后的数据。

2、由于某些原因，事务A并没有完成提交，发生了RollBack操作，则事务B读取的数据就是脏数据。

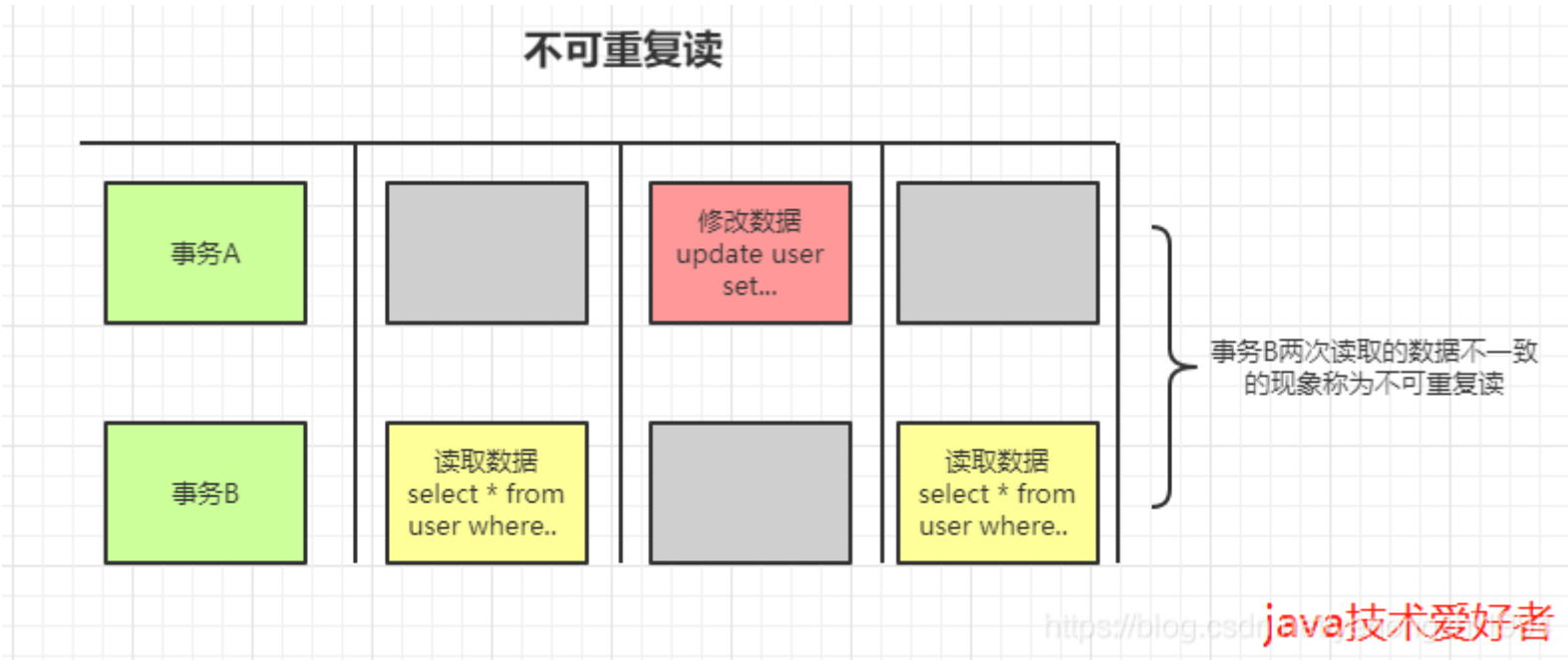
这种**读取到另一个事务未提交的数据的现象就是脏读(Dirty Read)。**



不可重复读

事务B读取了两次数据资源，在这两次读取的过程中事务A修改了数据，导致事务B在这两次读取出来的数据不一致。

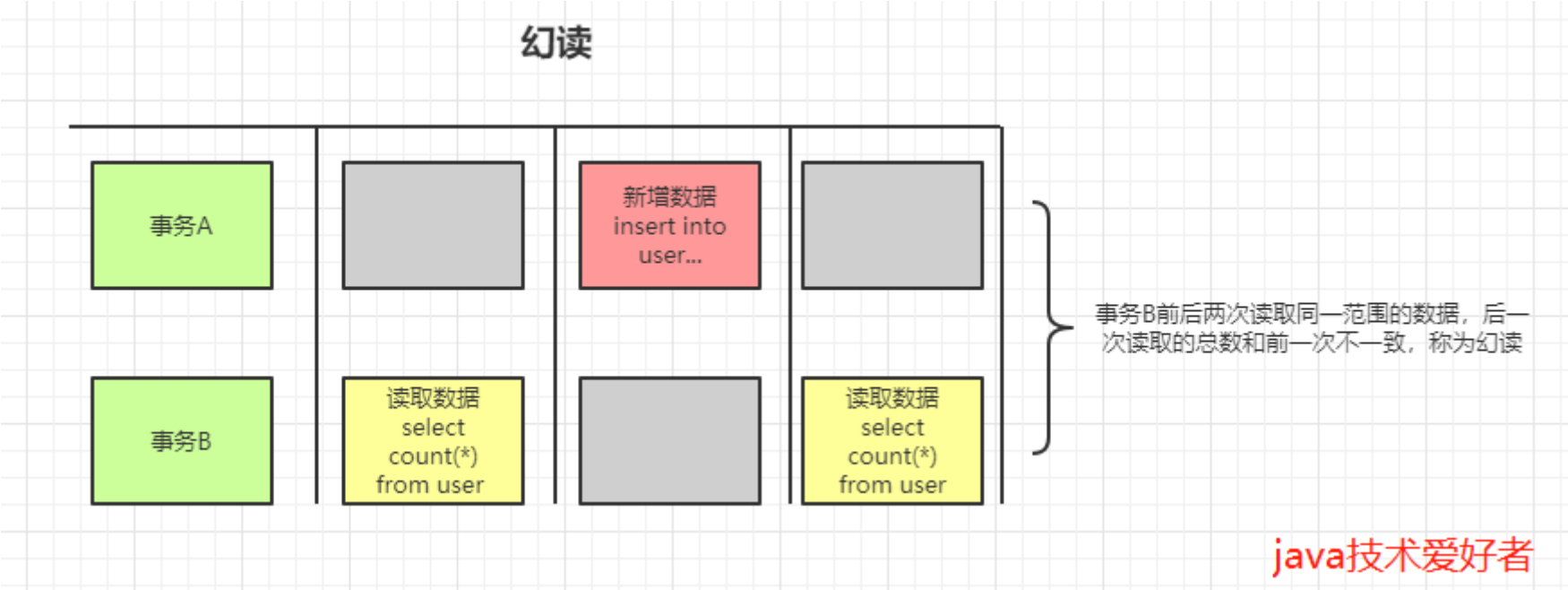
这种**在同一个事务中，前后两次读取的数据不一致的现象就是不可重复读(Nonrepeatable Read)。**



虚读(幻读)

事务B前后两次读取**同一个范围的数据**，在事务B两次读取的过程中事务A新增了数据，导致事务B后一次读取到前一次查询没有看到的行。

幻读和不可重复读有些类似，但是**幻读强调的是集合的增减，而不是单条数据的更新。**



第一类丢失更新(回滚丢失)

事务A和事务B都对数据进行更新，但是事务A由于某种原因事务回滚了，把已经提交的事务B的更新数据给覆盖了。这种现象就是第一类更新丢失。

时间	取款事务A	转账事务B
T1	**开始事务**	
T2		**开始事务**
T3	查询账户余额为1000元	
T4		查询账户余额为1000元
T5		汇入100元把余额改为1100元
T6		**提交事务**
T7	取出100元把余额改为900元	
T8	**撤销事务**	
T9	**余额恢复为1000 元（丢失更新）**	

https://blog.csdn.net @51CTO博客

第二类丢失更新(覆盖丢失/两次更新问题)

其实跟第一类更新丢失有点类似，也是两个事务同时对数据进行更新，但是事务A的更新把已提交的事务B的更新数据给覆盖了。这种现象就是第二类更新丢失。

时间	转账事务A	取款事务B
T1		**开始事务**
T2	**开始事务**	
T3		查询账户余额为1000元
T4	查询账户余额为1000元	
T5		取出100元把余额改为900元
T6		**提交事务**
T7	汇入100元	
T8	**提交事务**	
T9	**把余额改为1100 元（丢失更新）**	

https://blog.csdn.net @51CTO博客

事务隔离级别

为了解决以上的问题，主流的关系型数据库都会提供四种事务的隔离级别。 **事务隔离级别从低到高分别是：读未提交，读已提交，可重复读，串行化。事务隔离级别越高，越能保证数据的一致性和完整性，但是执行效率也越低，所以在设置数据库的事务隔离级别时需要做一下权衡，mysql默认是可重复读**

读未提交

读未提交(Read Uncommitted)，是最低的隔离级别， **所有的事务都可以看到其他未提交的事务的执行结果。** 只能防止第一类更新丢失，不能解决脏读，可重复读，幻读，所以很少应用于实际项目。

读已提交

读已提交(Read Committed)，在该隔离级别下，**一个事务的更新操作只有在该事务提交之后，另外一个事务才可能读取到同一笔数据更新后的结果。** 可以防止脏读和第一类更新丢失，但是不能解决可重复和幻读的问题。

可重复读（重要）

可重复读(Repeatable Read)，mysql默认的隔离级别。在该隔离级别下，**一个事务多次读同一个数据，在这个事务还没有结束时，其他事务不能访问该数据（包括了读写）**，这样就可以在同一个事务内两次读到的数据是一样的。可以防止脏读、不可重复读、第一类更新丢失，第二类更新丢失的问题，不过还是会出现幻读。

串行化

串行化(Serializable)，这是最高的隔离级别。**它要求事务序列化执行，事务只能一个接着一个的执行，不能并发执行。** 在这个级别，可以解决上面提到的所有并发问题，但是可能导致大量的超时现象和锁竞争，通常不会用这个隔离级别。

隔离级别	脏　读	不可重复读	幻　象　读	第一类丢失更新	第二类丢失更新
READ UNCOMMITED	允许	允许	允许	不允许	允许
READ COMMITTED	不允许	允许	允许	不允许	允许
REPEATABLE READ	不允许	不允许	允许	不允许	不允许
SERIALIZABLE	不允许	不允许	不允许	不允许	不允许

回滚机制

在mysql中，恢复机制是通过回滚日志（undo log）实现的，所有的事务进行的修改都会先记录到这个回滚日志中，然后在堆数据库中的对应进行写入。

mysql的事务是由redo和undo的，redo操作的所有信息都是记录到重做日志（redo_log）中，也就是说当一个事务做commit操作时，需要先把这个事务的操作写到redo_log中，然后在把这些操作flush到磁盘上，当出现故障时，只需要读取redo_log，然后在重新flush到磁盘就行了。

而对于undo就比较麻烦，mysql在处理事务时，会在数据共享表空间里申请一个段就做segment段，用保存undo信息，当在处理rollback，不是完完全全的物理undo，而是**逻辑undo**，也就是说会之前的操作进行反操作（对于每个insert，回滚时会执行delete；对于每个delete，回滚时会执行insert；对于每个update，回滚时会执行一个相反的update，把数据改回去。），但是这些共享表空间是不进行回收的。这些表空间的回收需要由mysql的master thread进程进行回收。

测试事务实现转账

执行事务

```sql
-- ============================================================

-- mysql 是默认开启事务自动提交的
set autocommit = 0; /* 关闭 */
set autocommit = 1; /* 开启（默认的） */

-- 手动处理事务
set autocommit = 0; -- 关闭自动条件

-- 事务开启
start transaction; -- 标记一个事务的开始，从这个之后的sql都在同一个事务内

insert xx;
insert xx;


-- 提交：持久化 （成功！）
commit;

-- 回滚：回到的原来的样子（失败！）
rollback;

-- 事务结束
set autocommit = 1; -- 开启自动提交

-- 了解
savepoint 保存点名 -- 设置一个事务的保存点
rollback to savepoint 保存点名 -- 回滚到保存点
release savepoint 保存点名 -- 撤销保存点
```

模拟场景

```sql
-- 转账
create database shop default character set utf8mb4 collate utf8mb4_unicode_ci;

use shop;

```

```sql
create table account(
    id int not null auto_increment,
    name varchar(30) not null,
    money decimal(9,2) not null,
    primary key (id)
)engine=innodb default charset=utf8;

insert into account(name,money)
values ("a",2000.00),("b",10000.00);

-- 模拟转账：事务
set autocommit = 0; -- 关闭自动提交

start transaction;  -- 开启事务

update shop.account set money=money-500 where name = "a"; -- a花掉500
update shop.account set money=money+500 where name = "b"; -- b赚了500

commit;  -- 提交事务（成功）

rollback; -- 回滚

set autocommit = 1; -- 恢复默认值
```

索引

什么是索引

Msql官方对索引的定义为：**索引（index）是帮助MySQL高效获取数据的数据结构。**提取句子主干，就可以得到索引的本质：索引是数据结构。

索引分类

在一个表中，主键索引只能有一个，唯一索引可以有多个

- 主键索引（primary key）
  - 唯一的标识，主键不可重复，只能有一个列作为主键
- 唯一索引 （unique key）
  - 避免重复的列出现，可以重复，多个列都可以标示为唯一索引
- 常规索引（key/index）
  - 默认的 index 或者key关键字来设置
- 全文索引（FullText）
  - 快速定位数据

创建索引

```sql
-- 索引的使用

-- 1.在创建表的时候给字段增加索引
-- 2.创建完毕后，增加索引

-- 显示所有的索引信息
SHOW INDEX FROM student;

-- 创建索引
create index 习 on 表名(列名)

-- 新增一个索引 (索引名) 列名

ALTER TABLE `student` ADD UNIQUE KEY `UK_IDENTITY_CARD` (`identity_card`);
ALTER TABLE `student` ADD KEY `K_STUDENT_NAME`(`student_name`);
ALTER TABLE `student`  ADD FULLTEXT INDEX `FI_PHONE` (`phone`);

-- explain 分析sql执行的状况

EXPLAIN SELECT * FROM student; -- 非全文索引

EXPLAIN SELECT * FROM student WHERE MATCH(`phone`) AGAINST('138'); -- 全文索引
```

SQL编程创建100万条数据测试索引

- 创建数据库

```sql
CREATE TABLE app_user (
  `id` BIGINT(20) UNSIGNED NOT NULL AUTO_INCREMENT COMMENT 'ID',
  `name` VARCHAR(50)  DEFAULT '' COMMENT '用户昵称',
  `email` VARCHAR(50)  NOT NULL COMMENT '用户邮箱',
  `phone` VARCHAR(20)  DEFAULT '' COMMENT '手机号',
  `gender` TINYINT(4)  UNSIGNED DEFAULT '0' COMMENT '性别（0: 男  1: 女）',
  `password` VARCHAR(100)  NOT NULL COMMENT '密码',
  `age` TINYINT(4)  DEFAULT '0' COMMENT '年龄',
  `create_time` DATETIME  DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
  `update_time` TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP COMMENT '更新时间',
  PRIMARY KEY (`id`)
) ENGINE=INNODB DEFAULT CHARSET=utf8mb4 COMMENT='app用户表';
```

- 添加100万条数据

```sql
SHOW VARIABLES LIKE 'log_bin_trust_function_creators';
SET @@global.log_bin_trust_function_creators='On';

-- 插入100万数据b (函数)

DELIMITER $$ -- 写函数之前必须要写，标志
CREATE FUNCTION mock_data()
RETURNS INT
BEGIN
  DECLARE num INT DEFAULT 1000000;
  DECLARE i INT DEFAULT 0;
  WHILE i<num DO
    INSERT INTO app_user(`name`,`email`,`phone`,`gender`,`password`,`age`)
    VALUES(CONCAT('用户',i),'123345@qq.com',CONCAT('18',FLOOR(RAND()*((999999999-100000000)+100000000))),FLOOR(RAND()*2),UUID(),FLOOR(RAND()*100));
    SET i = i+1;
  END WHILE;
  RETURN i;
END;

-- 执行函数
SELECT mock_data();
```

- 查看数据数目

```sql
SELECT COUNT(*) FROM app_user;
```

- 测试，加索引前

```sql
select * from app_user where name = "用户9999"; -- 0.440 sec
explain select * from app_user where name = "用户9999";
```

信息 | 结果 1 | 结果 2 | 剖析 | 状态

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered
1	SIMPLE	app_user	(Null)	ALL	(Null)	(Null)	(Null)	(Null)	992975	

- 测试，加索引后

```sql
-- 创建索引
create index id_app_user_name on app_user(name);

select * from app_user where name = "用户9999"; -- 0.001s
explain select * from app_user where name = "用户9999";
```

信息 | 结果 1 | 结果 2 | 剖析 | 状态

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered
1	SIMPLE	app_user	(Null)	ref	id_app_user_name	id_app_u	203	const	1	

索引在小数据量的时候，用处不大，但是再大数据的时候，区分十分明显

索引优化规则

- 索引不是越多越好
- 不要对经常变动的数据加索引
- 小数据量的表不需要加索引
- 索引一般加载常用来查询的字段上

- **负向条件查询不能使用索引**
  - 负向条件有：!=、<>、not in、not exists、not like 等。
- **不能使用索引中范围条件右边的列(范围列可以用到索引)，范围列之后列的索引全失效**

    - 范围条件有：<、<=、>、>=、between等。
    - 索引最多用于一个范围列，如果查询条件中有两个范围列则无法全用到索引。
- **不要在索引列上面做任何操作(计算、函数)，否则会导致索引失效而转向全表扫描**
- **强制类型转换会全表扫描**
- **利用延迟关联或者子查询优化超多分页场景**
    - 示例如下，先快速定位需要获取的id段，然后再关联:

```
selecta.* from 表1 a,(select id from 表1 where 条件 limit100000,20 ) b where a.id=b.id;
```

> 索引的数据结构

Hash类型的索引

bree ： innodb的默认数据结构

[CodingLabs - MySQL索引背后的数据结构及算法原理]

权限管理和备份

权限管理

可视化操作

示例：navicat

命令操作

MySQL的用户表： `mysql:user`

```
use mysql;
```

- 创建用户

```
-- 创建用户
create user hutao identified by "123456";
```

- 修改密码

```
-- 修改密码（修改当前用户密码）
set password for = "123456";

-- 修改密码（修改指定用户密码）
set password for hutao = "123456";
```

- 修改用户名

```
-- 重命名 rename user 原用户名 to 新用户名
rename user hutao to dahutao;
```

- 赋予用户权限

```
-- 用户授权  all privileges 全部的权限，库，表
-- all privileges 除了给别人授权不行，其他都能干

grant all privileges on *.* to dahutao;
```

- 查看权限

```sql
-- 查询权限
show grants for dahutao; -- 查询指定用户权限

show grants for root@localhost; -- 查看root用户的权限
```

- 撤销权限

```sql
-- 撤销权限 revoke 赋予的权限 在哪个库 给谁撤销
revoke all privileges on *.* from dahutao;
```

- 删除用户

```sql
-- 删除用户
drop user dahutao;
```

MySQL备份

为什么要备份?

- 保证重要的数据不丢失
- 数据转移

mysql数据库备份的方式

- 直接拷贝物理文件
- 在navicat这种可视化工具中手动导出sql文件
- 使用命令行导出 mysqldump 命令行使用

```sql
# 一张表 mysqldump -h 主机 -u 用户名 -p 密码 数据库 表名 >物理磁盘位置/文件名
mysqldump -h localhost -u root -p 123456 school student >D:/a.sql

# 多张表 mysqldump -h 主机 -u 用户名 -p 密码 数据库 表名1 表名2 >物理磁盘位置/文件名
mysqldump -h localhost -u root -p 123456 school student result >D:/a.sql

# 数据库 mysqldump -h 主机 -u用户名 -p密码 数据库 >物理磁盘位置/文件名
mysqldump -h localhost -u root -p 123456 school >D:/a.sql

# 导入
# 登录的情况下，切换到指定的数据库
# source 备份文件
use 数据库名称;
source 备份文件

# 也可以这样
mysql -u 用户名 -p 密码 库名<备份文件
```

规范数据库设计

为什么需要数据规范化?

- 信息重复
- 更新异常
- 插入异常：无法正常显示信息
- 删除异常：丢失有效的信息

第一范式（1NF）

1NF是对属性的 `原子性` ，要求属性具有原子性，不可再分解；

如学生（学号，姓名，性别，出生年月日），如果认为最后一列还可以再分成（出生年，出生月，出生日），它就不是一范式了，否则就是；

第二范式（2NF）

满足第二范式（2NF）必须先满足第一范式（1NF）。

2NF是对记录的 `唯一性` ，要求记录有唯一标识，即实体的唯一性，即 **不存在部分依赖**；

> 表：学号、课程号、姓名、学分;

这个表明显说明了两个事务:学生信息, 课程信息;由于**非主键字段必须依赖主键**，这里**学分依赖课程号**， **姓名依赖与学号**，所以不符合二范式。

可能会存在问题：

- `数据冗余：` 每条记录都含有相同信息；
- `删除异常：` 删除所有学生成绩，就把课程信息全删除了；
- `插入异常：` 学生未选课，无法记录进数据库；
- `更新异常：` 调整课程学分，所有行都调整。

正确做法:
学生： `Student` (学号, 姓名)；
课程： `Course` (课程号, 学分)；
选课关系： `StudentCourse` (学号, 课程号, 成绩)。

第三范式（3NF）

前提：满足第一范式和第二范式

3NF是对字段的 `冗余性` ，要求任何字段不能由其他字段派生出来，它要求字段没有冗余，即 **不存在传递依赖**；

> 表: 学号, 姓名, 年龄, 学院名称, 学院电话

因为存在 **依赖传递**: (学号) → (学生)→(所在学院) → (学院电话) 。

可能会存在问题:

- `数据冗余:` 有重复值;
- `更新异常:` 有重复的冗余信息，修改时需要同时修改多条记录，否则会出现 **数据不一致的情况** 。

正确做法:

学生：(学号, 姓名, 年龄, 所在学院)；

学院：(所在学院, 电话)。

反范式化

一般说来，数据库只需满足第三范式（ `3NF` ）就行了。

没有冗余的数据库设计可以做到。但是，没有冗余的数据库未必是最好的数据库，有时为了提高运行效率，就必须降低范式标准，适当保留冗余数据。具体做法是：在概念数据模型设计时遵守第三范式，降低范式标准的工作放到物理数据模型设计时考虑。降低范式就是增加字段，允许冗余， `达到以空间换时间的目的` 。

〖例〗：如订单表，“金额”这个字段的存在，表明该表的设计不满足第三范式，因为“金额”可以由“单价”乘以“数量”得到，说明“金额”是冗余字段。但是，增加“金额”这个冗余字段，可以提高查询统计的速度，这就是以空间换时间的作法。

规范性和性能的问题

关联查询的表不得超过三张表

- 考虑商业化的需求和目标（成本，用户体验）数据库的性能更加重要
- 在规范性能的问题的时候，需要适当的考虑一下规范性
- 故意给某些表增加一些冗余的字段。（从多表查询中变为单表查询）
- 故意增加一些计算列（从大数据库降低为小数据量的查询：索引）